

NOI 模拟赛

第一试

sz100309

时间：07:30 ~ 12:30

题目名称	区间	树	字符串
题目类型	传统型	传统型	传统型
目录	interval	tree	string
可执行文件名	interval	tree	string
输入文件名	interval.in	tree.in	string.in
输出文件名	interval.out	tree.out	string.out
每个测试点时限	2.0 秒	2.5 秒	2.0 秒
内存限制	250 MiB	1000 MiB	250 MiB
测试包数量	8	8	8
测试包是否等分	否	否	否

提交源程序文件名

对于 C++ 语言	interval.cpp	tree.cpp	string.cpp
-----------	--------------	----------	------------

编译选项

对于 C++ 语言	-O2 -std=c++14 -static
-----------	------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 main() 返回值类型必须为 int，程序正常结束时的返回值必须是 0。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
4. 选手提交的程序源文件必须不大于 100KB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 只提供 Linux 格式附加样例。
7. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

区间 (interval)

原题

若区间 $[a, b]$ 直接可达 $[c, d]$ ，则连单向边。

若 $[a, b]$ 和 $[c, d]$ 部分相交则属于同一 SCC，因此一个 SCC 对应一个区间。

显然 SCC 之间的可达性也有传递性，因此一次查询 (x, y) 等价于查询 x 所属 SCC 与 y 所属 SCC 是否有可达关系。

设 x 属于 $[a, b]$ ，而 y 属于 $[c, d]$ ，则可达当且仅当 $[a, b] \subseteq [c, d]$ 。

用并查集维护 SCC，保存每个连通块的左端点最小值和右端点最大值，问题转化为如何快速找出需要合并的对。

由于插入区间的长度递增，因此不会出现后插入的成为先插入的子区间，插入一个区间 $[l, r]$ 时，需要找出所有它之前的、包含 l 位置或 r 位置的区间（显然实际上它们与 $[l, r]$ 部分相交），并将它们合并。

维护一棵线段树，每个区间保存结点的列表，表示若 l 或 r 在当前区间中，则需要与列表中的结点所在连通块合并。

每加入一个 SCC 范围为 $[l, r]$ 时，**取出**线段树上递归到 l 或 r 时经过的所有结点的列表，将它们合并后，在线段树的 $[l + 1, r - 1]$ 区间中插入。

时间复杂度 $O(n \log n \alpha(n))$ 。

参考

树 (tree)

原题

令 s_u 表示子树 u 内 w 之和。

显然结点 u 要满流需要 $k_u = s_u$ 的流量,而向上推其父亲需要 $k_{f_u} = \sum_{v \in \text{son}(f_u)} \min(s_v, k_u)$, 直接计算容易做到 $O(n^2)$ 。

对于一个 u , 定义关键点为 u 到根的路径上满足存在至少两个儿子 $s_v \geq k_u$ 的点, 显然其数量不超过 $O(\sum_u w_u)$, 若每次可以快速找到下一个关键点, 则可以保证合理的复杂度。

考虑倍增, 令 $kt_{i,u}$ 表示从 u 向上走 2^i 步都不存在关键点, k_u 的最小值。

令 $sm_{i,u}$ 表示 u 及其 $1 \sim 2^i - 1$ 级祖先的兄弟的 w 之和, 令 $kf_{i,u}$ 表示 u 的 2^i 级祖先, 两者的处理是容易的。

$kt_{0,u}$ 为 u 的兄弟的 w_u 的最大值, 容易预处理。

显然 $kt_{i,u} = \max(kt_{i-1,u}, kt_{i-1,kf_{i-1,u}} - sm_{i-1,u})$, 因此可以 $O(n \log n)$ 预处理 kt 。

对于一个 u , 每次用 kt 向上跳 (若 $k \geq kt_{i,u}$ 则 k 加上 $sm_{i,u}$), 无法跳时暴力移到父亲。

时间复杂度 $O(n \log n \log(nw))$, 其中 $w = \max_u w_u$ 。

参考

字符串 (string)

原题

先考虑对于确定的 (s, t) 如何计算其权值。

当 $s = t$ 时, 显然方案数为 $(\sum_{i=1}^n 2^i)^2 = (2^{n+1} - 2)^2$, 以下假定 $s \neq t$ 。

定理. 合法的 a 和 b 有长度为 $\gcd(|a|, |b|)$ 的公共周期。

证明:

- 不妨设 $|a| \leq |b|$, 且 s 字典序小于 t 。
- 若 s 是 t 的前缀则必然无解, 否则显然存在位置 p 满足 $s_p \neq t_p$, 设 p 为满足此条件的最小值。
- 显然 $s_p = A$ 且 $t_p = B$ 。
- 此时可得出 a 是 b 的前缀, 若 $|a| = |b|$ 则 $a = b$ 满足命题, 否则重定义 b 为其长度 $|b| - |a|$ 的后缀, 并将所有 B 替换为 AB , 显然等价。
- 反复执行上述操作, 发现两串长度变化相当于辗转相减, 因此最终得到两者长度为 $\gcd(|a|, |b|)$ 的前缀, 且同时为两者的周期。

此时 s 和 t 中 A 和 B 的位置不重要了, 设 s 中 A 的数量减去 t 中 A 的数量为 d_a , t 中 B 的数量减去 s 中 B 的数量为 d_b , 则需要满足 $d_a \times |a| = d_b \times |b|$ 。

若 $d_a = d_b = 0$, 则对两者长度没有限制, 显然答案为 $\sum_{i=1}^n \sum_{j=1}^n 2^{\gcd(i,j)}$, 莫反得 $\sum_{d=1}^n \sum_{k=1}^{\lfloor \frac{n}{d} \rfloor} \mu(k) 2^d \lfloor \frac{n}{kd} \rfloor^2$, 只和 n 有关, 容易 $O(n \ln n)$ 预处理。

否则若 $d_a \times d_b \leq 0$ 则必然无解。

若 $d_a \neq 0, d_b \neq 0$, 则 $|a| : |b| = d_b : d_a$, 设约分后为 $u : v$, 则 $\gcd(|a|, |b|) : |a| : |b| = 1 : u : v$, 而 $|a|, |b| \leq n$, 因此 $\gcd(|a|, |b|) \leq \frac{n}{\max(u,v)}$, 从而答案为 $\sum_{i=1}^{\lfloor \frac{n}{\max(u,v)} \rfloor} 2^i$, 容易 $O(1)$ 计算。

然后考虑 $?$ 的贡献。

令 $f(d_a, d_b)$ 表示前文算出的 (d_a, d_b) 的贡献, 单个容易做到 $O(\log V)$ (瓶颈在于 $\gcd$)。

先忽略 $s = t$ 的情况, 当 s 和 t 在 $? = A, ? = B$ 意义下相等时令最终答案加上 $2^c((2^{n+1} - 2)^2 - f(0, 0))$ 即可, 其中 c 表示上下都是 $?$ 的位置数量。

令 d_a, d_b 为忽略 $?$ 时计算出的结果, q_c 和 q_d 表示上下 $?$ 的数量, 则答案为 $\sum_{i=0}^{q_c} \sum_{j=0}^{q_d} \binom{q_c}{i} \binom{q_d}{j} f(d_a + i - j, d_b + (q_d - j) - (q_c - i))$ 。

可化为 $\sum_{k=d_a-q_d}^{d_a+q_c} f(k, k - d_a - q_c + d_b + q_d) \sum_{i=0}^{q_c} \binom{q_c}{i} \binom{q_d}{q_d - d_a + k - i}$, 而 $\sum_{i=0}^{q_c} \binom{q_c}{i} \binom{q_d}{q_d - d_a + k - i}$ 可以用范德蒙德卷积化简为 $\binom{q_c + q_d}{q_d - d_a + k}$ 。

总时间复杂度 $O((|s| + |t|) \log \max(|s|, |t|) + n \log n)$, 足以通过本题。

实际上还可以进一步优化:

$O(n \log n)$ 来自计算 $f(0, 0)$, 这部分可以数论分块优化到 $O(n)$ 。

$O((|s| + |t|) \log \max(|s|, |t|))$ 来自计算 gcd, 发现形如给定 l, r, d 对于每个 $l \leq i \leq r$ 求 $\gcd(i, i + d)$, 等于求 $\gcd(i, d)$, 其中 $|d|, r - l, |l|, |r|$ 都是 $O(|s| + |t|)$ 的, 而 $f_d(x) = \gcd(x, d)$ 是积性函数, 可以 $O(|s| + |t|)$ 预处理。

因此总复杂度可以做到 $O(|s| + |t| + n)$ 。

[参考](#)

总结

预计 $(\geq 37) + 54 + 17$ 。

订正要求 300。